Marwadi University	Marwadi University Faculty of Technology Department of Information and Communication Technology	
Subject: FPGA Based System Design (01CT0718)	Aim: To design and implement a real-time digital alarm clock system using programmable logic devices.	
Experiment No: 11	Date:	Enrolment No: 92301733013

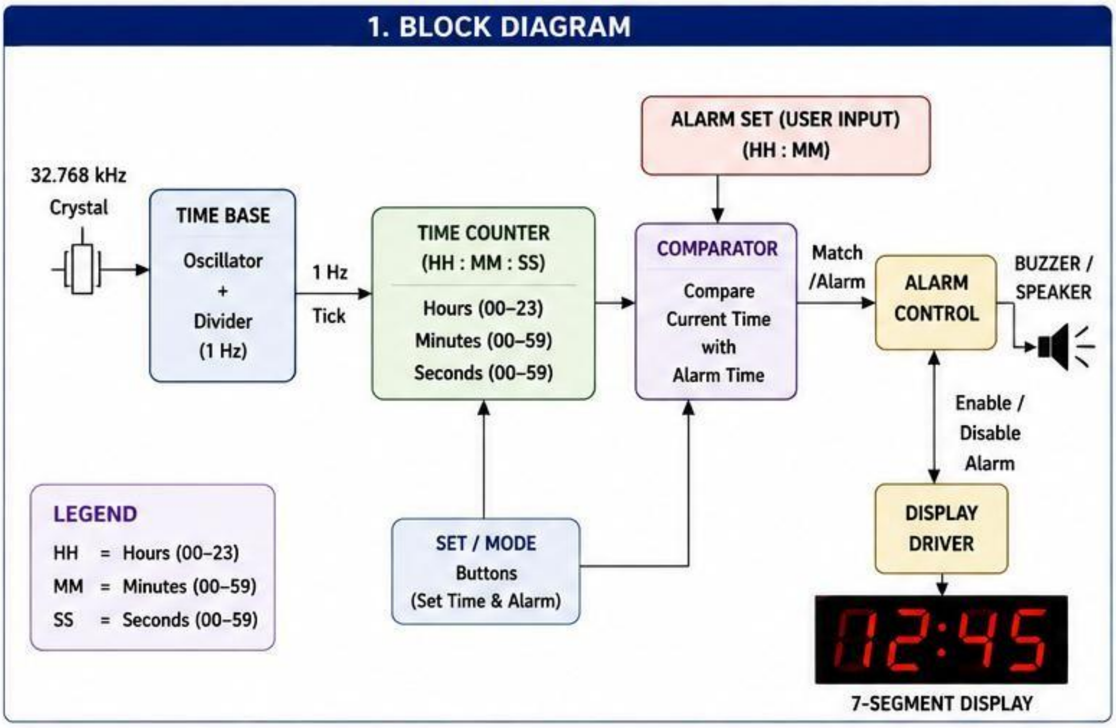
Aim: To design and implement a real-time digital alarm clock system using programmable logic devices.

Software required: Xilinx Vivado

Hardware required: Nexys4 DDR FPGA Development Board

Theory:

Block diagram:



Truth Table:

2. TRUTH TABLE (ALARM ACTIVATION LOGIC)

Current Time		Alarm Time		Comparison		Alarm Output (ALARM)
Hr (CH)	Min (CM)	Hr (AH)	Min (AM)	(CH = AH)	(CM = AM)	
0	0	0	0	1	1	1 (ON)
0	0	0	1	1	0	0 (OFF)
0	0	1	0	0	1	0 (OFF)
0	0	1	1	0	0	0 (OFF)
1	1	1	1	1	1	1 (ON)
1	1	1	2	1	0	0 (OFF)
1	2	1	1	1	0	0 (OFF)
1	2	1	2	1	1	1 (ON)

Note: Alarm output is 1 (ON) only when both Hour and Minute match.

Boolean Expression:

3. BOOLEAN EXPRESSIONS

COMPARATORS

E_H (Hour Match) = $(CH_{3} \text{ XNOR } AH_{3})(CH_{2} \text{ XNOR } AH_{2})(CH_{1} \text{ XNOR } AH_{1})(CH_{0} \text{ XNOR } AH_{0})$

E_M (Minute Match) = $(CM_{5} \text{ XNOR } AM_{5})(CM_{4} \text{ XNOR } AM_{4})(CM_{3} \text{ XNOR } AM_{3})$
$(CM_{2} \text{ XNOR } AM_{2})(CM_{1} \text{ XNOR } AM_{1})(CM_{0} \text{ XNOR } AM_{0})$

ALARM OUTPUT

$ALARM = E_H \bullet E_M$

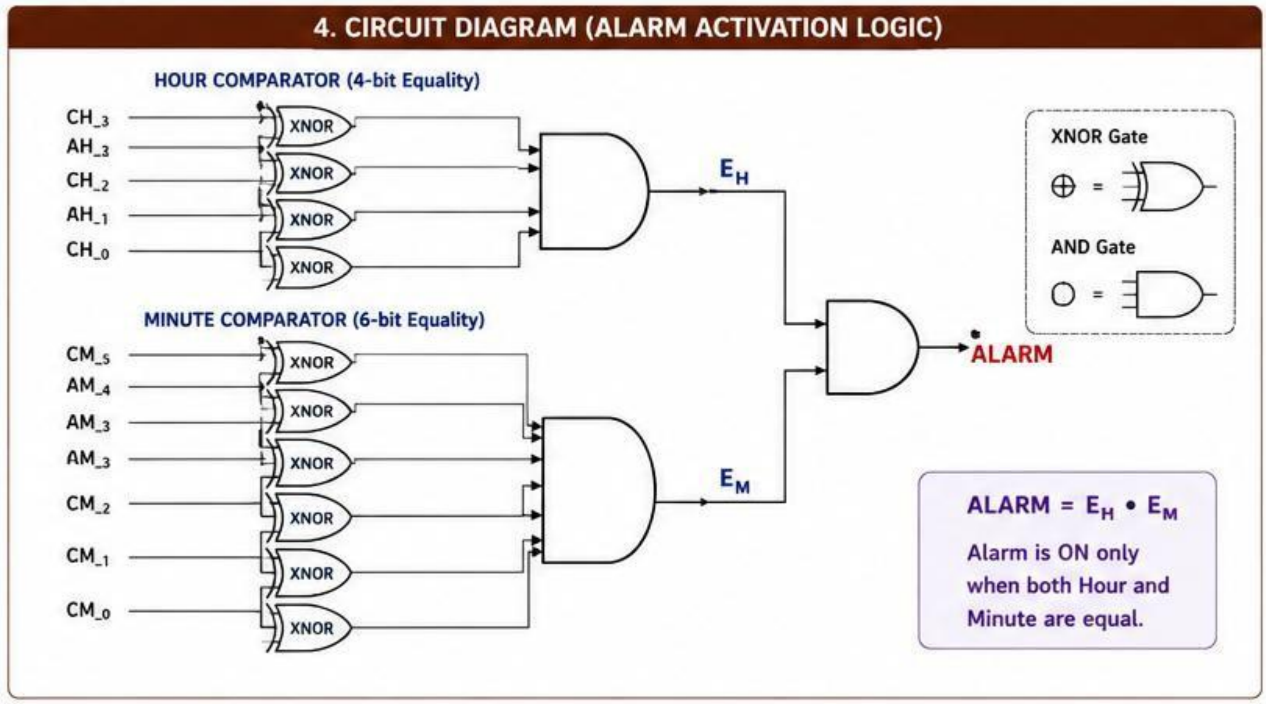
WHERE,

CH_i = i^{th} bit of Current Hour (i = 3 to 0) CM_i = i^{th} bit of Current Minute (i = 5 to 0)
AH_i = i^{th} bit of Alarm Hour (i = 3 to 0) AM_i = i^{th} bit of Alarm Minute (i = 5 to 0)

For n–bit equality: $A = B = (A_{n-1} \text{ XNOR } B_{n-1})(A_{n-2} \text{ XNOR } B_{n-2}) \ldots (A_0 \text{ XNOR } B_0)$

Circuit Diagram:

Marwadi University	Marwadi University Faculty of Technology Department of Information and Communication Technology	
Subject: FPGA Based System Design (01CT0718)	**Aim: To design and implement a real-time digital alarm clock system using programmable logic devices.**	
Experiment No: 11	**Date:**	**Enrolment No: 92301733013**



Verilog Code:

```
`timescale 1ns / 1ps

module risc_8 #(
    parameter CLK_FREQ_HZ = 100_000_000,
    parameter CPU_FREQ_HZ = 1
)(
    input wire CLK100MHZ,
    input wire BTNC,

    output reg [7:0] LED
);

    // Opcodes
    localparam OP_NOP  = 4'b0000;
    localparam OP_MOVI = 4'b0001;
    localparam OP_ADD  = 4'b0010;
    localparam OP_SUB  = 4'b0011;
    localparam OP_AND  = 4'b0100;
    localparam OP_OR   = 4'b0101;
    localparam OP_XOR  = 4'b0110;
    localparam OP_ADDI = 4'b0111;
    localparam OP_OUT  = 4'b1000;
    localparam OP_JMP  = 4'b1001;
    localparam OP_BEQZ = 4'b1010;
    localparam OP_HALT = 4'b1111;

//Registers
    reg [7:0] R0;
```

```
    reg [7:0] R1;
    reg [7:0] R2;
    reg [7:0] R3;

    reg [7:0] PC;

    reg [7:0] OUT_REG;

    reg HALTED;

    // 100 MHz -> CPU clock enable

    localparam integer CPU_DIV =
        CLK_FREQ_HZ / CPU_FREQ_HZ;

    reg [31:0] clk_counter;

    wire cpu_tick;

    assign cpu_tick = (clk_counter == CPU_DIV - 1);

    always @(posedge CLK100MHZ or posedge BTNC) begin

        if (BTNC)
            clk_counter <= 32'd0;

        else if (cpu_tick)
            clk_counter <= 32'd0;

        else
            clk_counter <= clk_counter + 1'b1;

    end

//Instruction ROM
    reg [15:0] instruction;

    always @(*) begin

        case (PC)

            8'd0:
                instruction = 16'b0001_00_00_00_000101;
                // MOVI R0, 5

            8'd1:
                instruction = 16'b0001_01_00_00_000011;
                // MOVI R1, 3

            8'd2:
                instruction = 16'b0010_10_00_01_000000;
                // ADD R2, R0, R1

            8'd3:
```

```
          instruction  =  16'b1000_10_00_00_000000;
          // OUT R2

        8'd4:
          instruction  =  16'b1111_00_00_00_000000;
          // HALT

        default:
          instruction = 16'b1111_00_00_00_000000;

      endcase

  end

//instruction fields
  wire [3:0] opcode;

  wire [1:0] rd;
  wire [1:0] rs1;
  wire [1:0] rs2;

  wire [5:0] imm6;

  assign  opcode = instruction[15:12];

  assign rd  = instruction[11:10];
  assign rs1 = instruction[9:8];
  assign rs2 = instruction[7:6];

  assign  imm6 = instruction[5:0];

 //read mux
  reg [7:0] rs1_data;
  reg [7:0] rs2_data;

  always @(*) begin

      case (rs1)

        2'b00:  rs1_data  =  R0;
        2'b01:  rs1_data  =  R1;
        2'b10:  rs1_data  =  R2;
        2'b11:  rs1_data = R3;

      endcase

  end

  always @(*) begin

      case (rs2)

        2'b00:  rs2_data  =  R0;
        2'b01:  rs2_data = R1;
```

```
        2'b10: rs2_data = R2;
        2'b11: rs2_data = R3;

      endcase

    end

  //ALU
    reg [7:0] alu_result;

    always @(*) begin

       case (opcode)

         OP_ADD:
            alu_result = rs1_data + rs2_data;

         OP_SUB:
            alu_result = rs1_data - rs2_data;

         OP_AND:
            alu_result = rs1_data & rs2_data;

         OP_OR:
            alu_result = rs1_data | rs2_data;

         OP_XOR:
            alu_result = rs1_data ^ rs2_data;

         OP_ADDI:
            alu_result = rs1_data + {{2{imm6[5]}}, imm6};

         default:
            alu_result = 8'd0;

      endcase

    end

//register write task
    task write_register;

       input  [1:0] register_number;
      input [7:0] data;

      begin

          case (register_number)

            2'b00:
               R0 <= data;

            2'b01:
               R1 <= data;
```

```
            2'b10:
                R2 <= data;

            2'b11:
                R3 <= data;

        endcase

    end

endtask

//CPU exeecution
always @(posedge CLK100MHZ  or posedge BTNC)  begin

    if (BTNC) begin

        R0      <= 8'd0;
        R1      <= 8'd0;
        R2      <= 8'd0;
        R3      <= 8'd0;

        PC      <= 8'd0;

        OUT_REG <= 8'd0;

        HALTED  <= 1'b0;

    end

    else if (cpu_tick && !HALTED) begin

        case (opcode)
//nop
            OP_NOP: begin
                PC <= PC + 1'b1;
            end

//mov1
            OP_MOVI: begin

                write_register(
                    rd,
                    {{2{imm6[5]}}, imm6}
                );

                PC <= PC + 1'b1;

            end

//add
            OP_ADD: begin
```

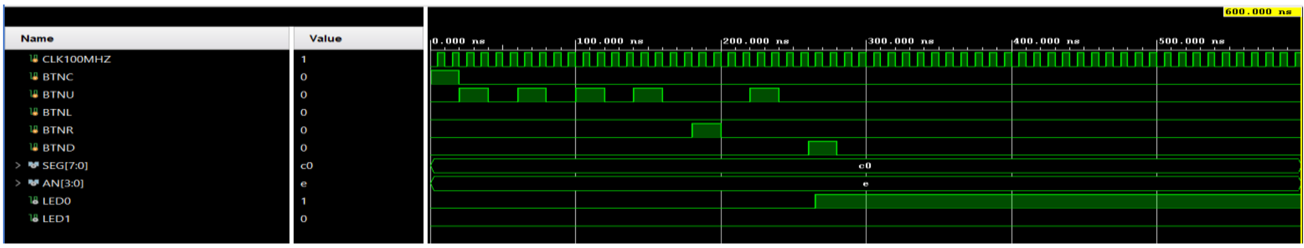
```
        write_register(
            rd,
            rs1_data + rs2_data
        );

        PC <= PC + 1'b1;

    end

    // SUB

    OP_SUB: begin

        write_register(
            rd,
            rs1_data - rs2_data
        );

        PC <= PC + 1'b1;

    end

//AND
    OP_AND: begin

        write_register(
            rd,
            rs1_data & rs2_data
        );

        PC <= PC + 1'b1;

    end


    //OR
    OP_OR: begin

        write_register(
            rd,
            rs1_data | rs2_data
        );

        PC <= PC + 1'b1;

    end

//XOR
    OP_XOR: begin

        write_register(
            rd,
            rs1_data ^ rs2_data
        );
```

```
        PC <= PC + 1'b1;

    end

    OP_ADDI: begin

        write_register(
            rd,
            rs1_data + {{2{imm6[5]}}, imm6}
        );

        PC <= PC + 1'b1;

    end

    OP_OUT: begin

        case (rd)

            2'b00:
                OUT_REG <= R0;

            2'b01:
                OUT_REG <= R1;

            2'b10:
                OUT_REG <= R2;

            2'b11:
                OUT_REG <= R3;

        endcase

        PC <= PC + 1'b1;

    end


   //JMP
    OP_JMP: begin

        PC <= {2'b00, imm6};

    end

//BEQZ
    OP_BEQZ: begin

        if (rs1_data == 8'd0)
            PC <= {2'b00, imm6};
        else
            PC <= PC + 1'b1;
```

```
            end

//Halt
            OP_HALT: begin

               HALTED <= 1'b1;

            end


            default:  begin

               PC <= PC + 1'b1;

            end

         endcase

      end

   end

//LEDs
   always @(*) begin
      LED = OUT_REG;
   end

endmodule
```

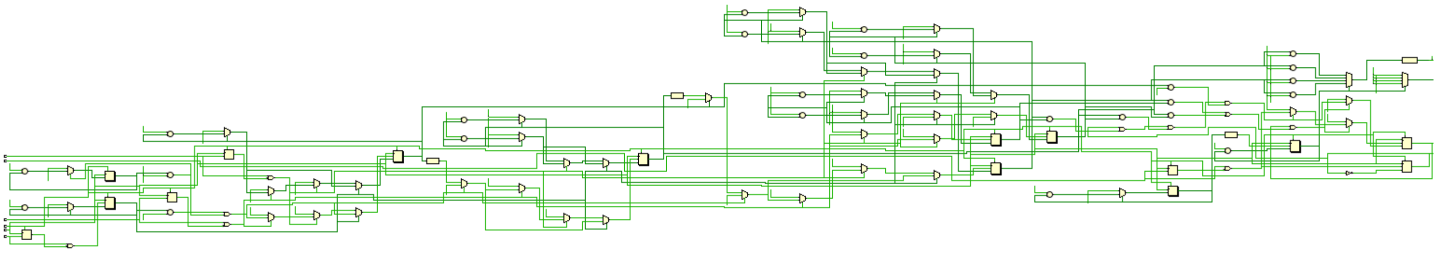
Simulation Results:

Waveform:

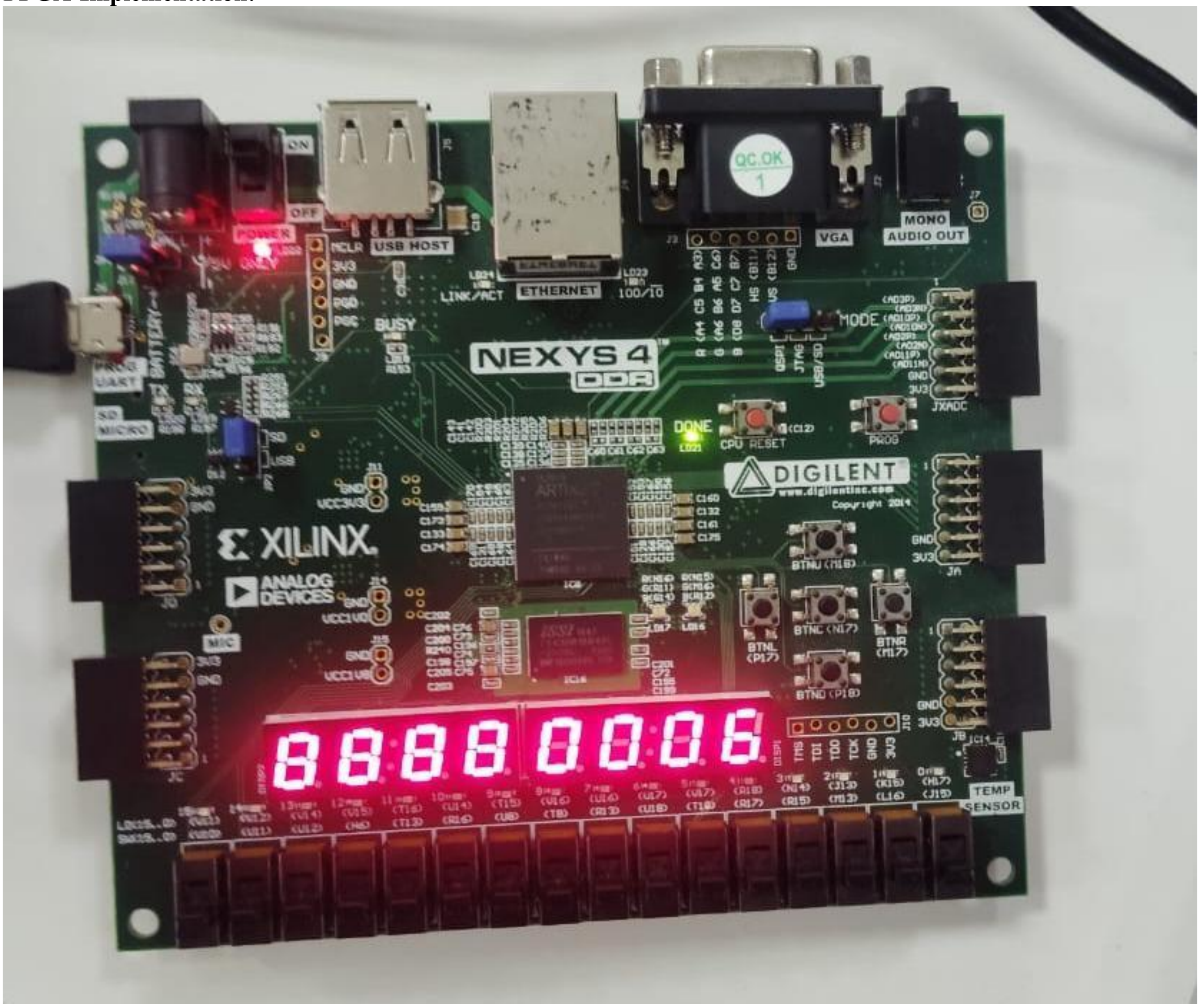


RTL Simulation:

Marwadi University	Marwadi University Faculty of Technology Department of Information and Communication Technology	
Subject: FPGA Based System Design (01CT0718)	Aim: To design and implement a real-time digital alarm clock system using programmable logic devices.	
Experiment No: 11	Date:	Enrolment No: 92301733013



FPGA Implementation:



Conclusion:

The real-time digital alarm clock system was successfully designed using Verilog HDL and implemented on the Nexys4 DDR FPGA board through Xilinx Vivado. The system combined a time-base generator, counter logic, user input controls, and comparator circuits to accurately track time and trigger the alarm. The design was verified using behavioral simulations, synthesized RTL schematics, and hardware testing with the 7-segment display and LEDs. Overall, the project demonstrated the effectiveness, speed, and flexibility of programmable logic devices for implementing real-time digital systems.

Post Lab Exercise:

1. Why is a programmable logic device suitable for implementing a digital alarm clock?

   A programmable logic device (PLD) offers a flexible and reliable way to implement digital circuits in hardware. Its design can be easily modified or reconfigured without the need to change the physical components.

2. How does a clock divider enable real-time clock operation?

   A clock divider converts a high-frequency system clock into a 1 Hz signal. This allows the clock to count seconds accurately in real time.

3. Why is synchronous design preferred for a digital alarm clock?

   In a synchronous design, all components operate according to a common clock signal, which helps minimize timing errors. This results in more stable, reliable, and predictable system operation.

4. How does a finite state machine improve alarm clock functionality?

   An FSM manages the different operating modes of the system, including time display, time setting, alarm setting, and alarm activation. This provides a structured approach to controlling the system and makes its operation easier to manage.

5. Why is debouncing required for push-button inputs?

   Mechanical push buttons can produce multiple unwanted signals due to contact bouncing when pressed. A debouncing circuit filters out these unwanted transitions so that each physical button press is recognized as a single valid input.

6. How does a reset circuit improve system reliability?

   A reset circuit initializes the system to a known starting state during power-up or faults. This prevents incorrect operation and improves reliability.

7. Why should the alarm operate independently of time-setting mode?

   The system should continue tracking the current time while the user is setting the clock. This allows the alarm function to keep monitoring the time and ensures that alarms are triggered accurately without interruption.

8. Why is BCD representation commonly used in digital clocks?

   BCD (Binary-Coded Decimal) represents each decimal digit separately, making it easy to drive seven-segment displays. It also simplifies time calculations and display logic.